

requirements.txt

Flask==3.0.0

Flask-SQLAlchemy==3.1.1

email-validator==2.1.0.post1

(If you want, you can add python-dotenv later for environment variables.)

2 config.py

import os

BASE_DIR = os.path.abspath(os.path.dirname(__file__))

class Config:

SECRET_KEY = os.environ.get("SECRET_KEY", "dev-secret-change-me")

SQLALCHEMY_DATABASE_URI = (

os.environ.get("DATABASE_URL")

or "sqlite:/// " + os.path.join(BASE_DIR, "instance", "phishing.db")

)

SQLALCHEMY_TRACK_MODIFICATIONS = False

```
# SMTP settings – for lab use only

SMTP_HOST = os.environ.get("SMTP_HOST", "localhost")
SMTP_PORT = int(os.environ.get("SMTP_PORT", 25))
SMTP_USE_TLS = bool(int(os.environ.get("SMTP_USE_TLS", "0")))
SMTP_USERNAME = os.environ.get("SMTP_USERNAME") # optional
SMTP_PASSWORD = os.environ.get("SMTP_PASSWORD") # optional
SMTP_FROM = os.environ.get("SMTP_FROM", "no-reply@phishing-lab.local")
```

❏ models.py

```
from datetime import datetime
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy.orm import relationship
import uuid
```

```
db = SQLAlchemy()
```

```
def generate_token():
    return uuid.uuid4().hex
```

```
class Campaign(db.Model):
```

```
__tablename__ = "campaigns"
```

```
id = db.Column(db.Integer, primary_key=True)
```

```
name = db.Column(db.String(120), nullable=False)
```

```
description = db.Column(db.Text, nullable=True)
```

```
subject = db.Column(db.String(200), nullable=False)
```

```
email_body = db.Column(db.Text, nullable=False)
```

```
landing_message = db.Column(db.Text, nullable=False, default="This was a simulation.")
```

```
created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

```
targets = relationship("Target", back_populates="campaign", cascade="all, delete-orphan")
```

```
def total_targets(self):
```

```
    return len(self.targets)
```

```
def total_sent(self):
```

```
    return sum(1 for t in self.targets if t.sent_at is not None)
```

```
def total_clicked(self):
```

```
    return sum(1 for t in self.targets if t.clicked_at is not None)
```

```
def total_submitted(self):
```

```
    return sum(1 for t in self.targets if t.submitted_at is not None)
```

```
def click_rate(self):
```

```
if self.total_targets() == 0:
    return 0.0
return (self.total_clicked() / self.total_targets()) * 100.0
```

```
def success_rate(self):
    if self.total_targets() == 0:
        return 0.0
    return (self.total_submitted() / self.total_targets()) * 100.0
```

```
class Target(db.Model):
```

```
    __tablename__ = "targets"
```

```
    id = db.Column(db.Integer, primary_key=True)
```

```
    email = db.Column(db.String(200), nullable=False)
```

```
    token = db.Column(db.String(64), unique=True, default=generate_token)
```

```
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

```
    sent_at = db.Column(db.DateTime, nullable=True)
```

```
    clicked_at = db.Column(db.DateTime, nullable=True)
```

```
    submitted_at = db.Column(db.DateTime, nullable=True)
```

```
    ip_address = db.Column(db.String(64), nullable=True)
```

```
    user_agent = db.Column(db.String(300), nullable=True)
```

```
    submitted_data = db.Column(db.Text, nullable=True) # anonymized / dummy
```

```
    campaign_id = db.Column(db.Integer, db.ForeignKey("campaigns.id"), nullable=False)
```

```
    campaign = relationship("Campaign", back_populates="targets")
```

4email_utils.py

This module sends emails via the local SMTP (Postfix/Sendmail or lab SMTP server).

```
import smtplib

from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from flask import current_app


def send_phishing_email(to_email: str, subject: str, html_body: str):
    """
    Sends a single phishing simulation email.
    This must only be used in a controlled lab with informed consent.
    """

    cfg = current_app.config
    msg = MIMEMultipart("alternative")
    msg["Subject"] = subject
    msg["From"] = cfg["SMTP_FROM"]
    msg["To"] = to_email

    part = MIMEText(html_body, "html")
```

```
msg.attach(part)
```

```
with smtplib.SMTP(cfg["SMTP_HOST"], cfg["SMTP_PORT"]) as server:
```

```
    if cfg["SMTP_USE_TLS"]:
```

```
        server.starttls()
```

```
    if cfg.get("SMTP_USERNAME") and cfg.get("SMTP_PASSWORD"):
```

```
        server.login(cfg["SMTP_USERNAME"], cfg["SMTP_PASSWORD"])
```

```
    server.sendmail(cfg["SMTP_FROM"], [to_email], msg.as_string())
```

```
---
```

5 app.py (Main Flask App)

```
import os
```

```
from datetime import datetime
```

```
from flask import (
```

```
    Flask, render_template, request, redirect, url_for, flash, abort
```

```
)
```

```
from email_validator import validate_email, EmailNotValidError
```

```
from config import Config
```

```
from models import db, Campaign, Target
```

```
from email_utils import send_phishing_email
```

```
def create_app():

    app = Flask(__name__, instance_relative_config=True)
    app.config.from_object(Config)

    # Ensure instance folder exists (for SQLite DB)
    try:
        os.makedirs(app.instance_path, exist_ok=True)
    except OSError:
        pass

    db.init_app(app)

    @app.before_first_request
    def init_db():
        db.create_all()

    # ----- ROUTES -----

    @app.route("/")
    def index():
        campaigns = Campaign.query.order_by(Campaign.created_at.desc()).all()
        return render_template("index.html", campaigns=campaigns)

    @app.route("/campaign/new", methods=["GET", "POST"])
    def create_campaign():
        if request.method == "POST":
```

```
name = request.form.get("name", "").strip()
description = request.form.get("description", "").strip()
subject = request.form.get("subject", "").strip()
email_body = request.form.get("email_body", "").strip()
landing_message = request.form.get("landing_message", "").strip()
raw_targets = request.form.get("targets", "").strip()
```

```
if not name or not subject or not email_body or not raw_targets:
    flash("Name, subject, body, and targets are required.", "danger")
    return render_template("create_campaign.html")
```

```
target_emails = [e.strip() for e in raw_targets.split(",") if e.strip()]
cleaned_emails = []
```

```
for e in target_emails:
    try:
        valid = validate_email(e)
        cleaned_emails.append(valid.email)
    except EmailNotValidError:
        flash(f"Skipping invalid email: {e}", "warning")
```

```
if not cleaned_emails:
    flash("No valid target emails.", "danger")
    return render_template("create_campaign.html")
```

```
if not landing_message:
```


landing_message = "This was an ethical phishing simulation. Please review best practices."

```
campaign = Campaign(  
    name=name,  
    description=description,  
    subject=subject,  
    email_body=email_body,  
    landing_message=landing_message,  
)
```

```
db.session.add(campaign)
```

```
db.session.flush() # get id
```

```
for em in cleaned_emails:
```

```
    t = Target(email=em, campaign=campaign)
```

```
    db.session.add(t)
```

```
db.session.commit()
```

```
flash("Campaign created successfully. You can now send emails in the campaign  
detail page.", "success")
```

```
return redirect(url_for("campaign_detail", campaign_id=campaign.id))
```

```
return render_template("create_campaign.html")
```

```
@app.route("/campaign/<int:campaign_id>")
```

```
def campaign_detail(campaign_id):
```

```
    campaign = Campaign.query.get_or_404(campaign_id)
```

```

return render_template("campaign_detail.html", campaign=campaign)

@app.route("/campaign/<int:campaign_id>/send", methods=["POST"])
def send_campaign(campaign_id):
    campaign = Campaign.query.get_or_404(campaign_id)

    # Reminder: for lab/education only
    for target in campaign.targets:
        # build tracking URL to landing page
        link = url_for("landing", token=target.token, _external=True)
        body = campaign.email_body.replace("{{link}}", link)

    try:
        send_phishing_email(target.email, campaign.subject, body)
        target.sent_at = datetime.utcnow()
    except Exception as e:
        # For a real system you would log this server-side
        print(f"Error sending to {target.email}: {e}")

    db.session.commit()

    flash("Emails sent (or attempted). Check SMTP logs for details.", "info")
    return redirect(url_for("campaign_detail", campaign_id=campaign.id))

@app.route("/l/<token>", methods=["GET", "POST"])
def landing(token):
    target = Target.query.filter_by(token=token).first()

```

```
if not target:
    abort(404)

if request.method == "GET":
    if not target.clicked_at:
        target.clicked_at = datetime.utcnow()
        target.ip_address = request.remote_addr
        target.user_agent = request.headers.get("User-Agent", "")
        db.session.commit()
    return render_template("landing.html", campaign=target.campaign, target=target)

# POST: user "submits" data (for simulation only)
submitted = {
    "dummy_username": request.form.get("username", ""),
    "dummy_password": request.form.get("password", ""),
}
target.submitted_at = datetime.utcnow()
target.submitted_data = str(submitted)
db.session.commit()
return redirect(url_for("educate", token=target.token))

@app.route("/educate/<token>")
def educate(token):
    target = Target.query.filter_by(token=token).first()
    if not target:
        abort(404)
```

```
return render_template("educate.html", campaign=target.campaign, target=target)
```

```
return app
```

```
if __name__ == "__main__":
```

```
    app = create_app()
```

```
    app.run(debug=True)
```

```
---
```

6. Templates

templates/base.html

```
<!doctype html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="utf-8">
```

```
    <title>Ethical Phishing Simulation Platform</title>
```

```
    <link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">
```

```
</head>
```

```
<body>
```

```
<nav class="navbar">
```

```
    <div class="container">
```

```

<a href="{{ url_for('index') }}" class="brand">PhishSim</a>

<a href="{{ url_for('create_campaign') }}" class="nav-link">New Campaign</a>

</div>

</nav>

<main class="container">

{% with messages = get_flashed_messages(with_categories=true) %}

{% if messages %}

<div class="flash-container">

{% for category, message in messages %}

<div class="flash flash-{{ category }}">{{ message }}</div>

{% endfor %}

</div>

{% endif %}

{% endwith %}

{% block content %}{% endblock %}

</main>

<footer class="footer">

<div class="container">

<small>For educational use only – ethical phishing simulations in a controlled
lab.</small>

</div>

</footer>

</body>

```

</html>

templates/index.html

{% extends "base.html" %}

{% block content %}

<h1>Campaigns</h1>

Create New Campaign

{% if campaigns %}

<table class="table">

<thead>

<tr>

<th>Name</th>

<th>Created</th>

<th>Targets</th>

<th>Clicks</th>

<th>Success Rate</th>

<th></th>

</tr>

</thead>

<tbody>

```

{% for c in campaigns %}

<tr>

<td>{{ c.name }}</td>

<td>{{ c.created_at.strftime('%Y-%m-%d %H:%M') }}</td>

<td>{{ c.total_targets() }}</td>

<td>{{ c.total_clicked() }}</td>

<td>{{ "%.1f" % c.success_rate() }}%</td>

<td><a href="{{ url_for('campaign_detail', campaign_id=c.id) }}">View</a></td>

</tr>

{% endfor %}

</tbody>

</table>

{% else %}

<p>No campaigns yet. Create one.</p>

{% endif %}

{% endblock %}

```

templates/create_campaign.html

```

{% extends "base.html" %}

{% block content %}

<h1>Create New Campaign</h1>

```

```
<form method="post" class="form">
```

```
<label>Campaign Name</label>
```

```
<input type="text" name="name" required>
```

```
<label>Description (optional)</label>
```

```
<textarea name="description" rows="3"></textarea>
```

```
<label>Email Subject</label>
```

```
<input type="text" name="subject" required>
```

```
<label>Email Body (HTML). Use <code>{{ '{{link}}' }}</code> where the phishing link should go.</label>
```

```
<textarea name="email_body" rows="8" required>
```

```
<p>Dear user,</p>
```

```
<p>There is an important update to your account. Please <a href="{{ '{{link}}' }}">click here</a> to review.</p>
```

```
<p>Thank you.</p>
```

```
</textarea>
```

```
<label>Landing Message (shown after click)</label>
```

```
<textarea name="landing_message" rows="4">
```

This was an ethical phishing simulation. You clicked on a simulated phishing email.

Please review the security tips to avoid real attacks in the future.

```
</textarea>
```

```
<label>Target Emails (comma-separated)</label>
```



```
<textarea name="targets" rows="4" required placeholder="user1@example.com,
user2@example.com"></textarea>
```

```
<button type="submit" class="btn">Create Campaign</button>
```

```
</form>
```

```
{% endblock %}
```

templates/campaign_detail.html

```
{% extends "base.html" %}
```

```
{% block content %}
```

```
<h1>Campaign: {{ campaign.name }}</h1>
```

```
<p>{{ campaign.description }}</p>
```

```
<div class="stats">
```

```
<div>Total Targets: <strong>{{ campaign.total_targets() }}</strong></div>
```

```
<div>Sent: <strong>{{ campaign.total_sent() }}</strong></div>
```

```
<div>Clicked: <strong>{{ campaign.total_clicked() }}</strong></div>
```

```
<div>Submitted: <strong>{{ campaign.total_submitted() }}</strong></div>
```

```
<div>Click Rate: <strong>{{ "%.1f" % campaign.click_rate() }}%</strong></div>
```

```
<div>Success Rate: <strong>{{ "%.1f" % campaign.success_rate() }}%</strong></div>
```

```
</div>
```

```
<form method="post" action="{{ url_for('send_campaign', campaign_id=campaign.id) }}">
    <button type="submit" class="btn">Send Emails (Lab Only)</button>
</form>
```

```
<h2>Targets</h2>
```

```
<table class="table">
```

```
    <thead>
```

```
        <tr>
```

```
            <th>Email</th>
```

```
            <th>Sent</th>
```

```
            <th>Clicked</th>
```

```
            <th>Submitted</th>
```

```
        </tr>
```

```
    </thead>
```

```
    <tbody>
```

```
        {% for t in campaign.targets %}
```

```
            <tr>
```

```
                <td>{{ t.email }}</td>
```

```
                <td>{{ t.sent_at or '-' }}</td>
```

```
                <td>{{ t.clicked_at or '-' }}</td>
```

```
                <td>{{ t.submitted_at or '-' }}</td>
```

```
            </tr>
```

```
        {% endfor %}
```

```
    </tbody>
```

```
</table>
```

```
{% endblock %}
```

templates/landing.html

```
{% extends "base.html" %}
```

```
{% block content %}
```

```
<h1>Sign in</h1>
```

```
<p>{{ campaign.landing_message }}</p>
```

```
<form method="post" class="form">
```

```
<label>Username (for simulation only)</label>
```

```
<input type="text" name="username">
```

```
<label>Password (for simulation only, do NOT use real password)</label>
```

```
<input type="password" name="password">
```

```
<button type="submit" class="btn">Submit</button>
```

```
</form>
```

```
<p class="hint">
```

```
  ⚠ This is part of a controlled training exercise. Do not enter real credentials.
```

```
</p>
```

```
{% endblock %}
```

templates/educate.html

{% extends "base.html" %}

{% block content %}

<h1>Security Awareness – Phishing Simulation</h1>

<p>Thank you for participating in this phishing awareness exercise.</p>

<h2>What happened?</h2>

You received a simulated phishing email as part of a training campaign.

You clicked on a link in the email, which led you to this training page.

<h2>How to spot phishing in the future</h2>

Check the sender address carefully.

Be suspicious of urgent or threatening language.

Hover over links to inspect the real URL before clicking.

Never enter your real password into unknown or suspicious pages.

Report suspicious emails to your security / IT team.

<p>If this were a real attack, an attacker could have captured your credentials or sensitive information.

Always pause and verify before clicking.</p>

Back to Dashboard

{% endblock %}

└─static/styles.css (simple styling)

```
body {  
    font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;  
    margin: 0;  
    background: #f5f5f7;  
    color: #222;  
}
```

```
.container {  
    width: 90%;  
    max-width: 960px;  
    margin: 0 auto;  
}
```

```
.navbar {
```

```
background: #111827;
color: #f9fafb;
padding: 0.75rem 0;
margin-bottom: 1rem;
}
```

```
.navbar .brand {
color: #f9fafb;
text-decoration: none;
font-weight: 600;
font-size: 1.1rem;
}
```

```
.navbar .nav-link {
margin-left: 1rem;
color: #e5e7eb;
text-decoration: none;
}
```

```
.footer {
margin-top: 2rem;
padding: 1rem 0;
text-align: center;
font-size: 0.85rem;
color: #6b7280;
}
```

```
h1, h2 {  
  color: #111827;  
}
```

```
.table {  
  width: 100%;  
  border-collapse: collapse;  
  margin-top: 1rem;  
  background: white;  
}
```

```
.table th, .table td {  
  border: 1px solid #e5e7eb;  
  padding: 0.5rem;  
  text-align: left;  
  font-size: 0.9rem;  
}
```

```
.table th {  
  background: #f3f4f6;  
}
```

```
.btn {  
  display: inline-block;  
  background: #2563eb;
```

```
color: #f9fafb;  
padding: 0.5rem 1rem;  
border-radius: 0.375rem;  
text-decoration: none;  
border: none;  
cursor: pointer;  
margin-top: 0.5rem;  
}
```

```
.btn:hover {  
  background: #1d4ed8;  
}
```

```
.form label {  
  display: block;  
  margin-top: 0.75rem;  
  font-weight: 500;  
}
```

```
.form input[type="text"],  
.form input[type="password"],  
.form textarea {  
  width: 100%;  
  padding: 0.4rem;  
  border-radius: 0.375rem;  
  border: 1px solid #d1d5db;
```



```
font-size: 0.9rem;
margin-top: 0.25rem;
box-sizing: border-box;
}
```

```
.flash-container {
  margin-bottom: 1rem;
}
```

```
.flash {
  padding: 0.5rem 0.75rem;
  border-radius: 0.375rem;
  margin-bottom: 0.25rem;
  font-size: 0.9rem;
}
```

```
.flash-success { background: #dcfce7; color: #166534; }
.flash-danger { background: #fee2e2; color: #991b1b; }
.flash-warning { background: #fef9c3; color: #854d0e; }
.flash-info { background: #e0f2fe; color: #075985; }
```

```
.stats {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(140px, 1fr));
  gap: 0.5rem;
  margin: 1rem 0;
```

```
}
```

```
.stats > div {  
  background: #fff;  
  padding: 0.5rem 0.75rem;  
  border-radius: 0.375rem;  
  border: 1px solid #e5e7eb;  
  font-size: 0.9rem;  
}
```

```
.hint {  
  margin-top: 1rem;  
  font-size: 0.85rem;  
  color: #6b7280;  
}
```

 How to run locally

1. Create & activate virtualenv (optional but recommended)

```
python -m venv venv
```

```
source venv/bin/activate # on Windows: venv\Scripts\activate
```

2. Install dependencies

```
pip install -r requirements.txt
```

3. Set optional SMTP env vars if needed, or use localhost

```
export SMTP_HOST=localhost
```

```
export SMTP_PORT=25
```

4. Run the app

```
python app.py
```

App will be at: <http://127.0.0.1:5000/>